

CSCI 232

232 Algorithms & Data Structures

LECTURE: 16

J. L. Pach

TODAY'S AGENDA

- References
  - Alias, `const T&`, return by reference, dangling references
  - Range-for, copy constructor, `T&&` and `std::move`
- Templates
  - Function templates, class templates, non-type parameters
  - Concepts, generic lambdas
- Standard containers
  - `vector`, `list`, `deque`, `map`, `unordered_map`, adapters
  - Iterators and `<algorithm>`
- Summary

REFERENCES

in C++

THREE WAYS TO PASS AN ARGUMENT

```
void swapVal(int a, int b)
{
  int t = a; a = b; b = t;
}     // copies
void swapPtr(int* a, int* b)
{
  int t = *a; *a = *b; *b = t;
}
void swapRef(int& a, int& b)
{
  int t = a; a = b; b = t;
}
```



- `swapVal(x, y)` swaps **copies**, so `x` and `y` stay unchanged.
- `swapPtr(&x, &y)` works, but the caller must write `&` and the callee `*`.
- `swapRef(x, y)` works and the call looks like a normal call.

WHAT IS A REFERENCE?

A reference is **another name** (an alias) for an existing object.

```
int x = 10;
int& r = x;          // r is an alias of x
r = 20;              // x is now 20
std::cout << (&r == &x);   // 1: the same address
```



- It **must be initialized** when declared: `int& r;` does not compile.
- It **cannot be reseated**: `r = y;` copies the value of `y` into `x`, it does not rebind `r`.
- There is no "null reference". The object always exists.

Reference vs Pointer

	Pointer `int*`	Reference `int&`
Can be `nullptr`	yes	no
Can be reassigned	yes	no
Must be initialized	no	yes
Access to the object	`*p`	`symbolic_name`
Typical use	optional object, own nodes (`new`)	parameters, aliases

> In your linked list the nodes are linked with **pointers**: a link can be `nullptr` and it changes over time. A reference can do neither.

CONST REFERENCE: NO COPY, NO CHANGE

```cpp
void print(const std::string& s)    // no copy of the string
{
    std::cout << s << "\n";
    // s += "x";                    // error: s is const
}

print("hello");                     // OK: binds to a temporary
```



- Passing `std::string` by value copies **all characters**: $\Theta(n)$ time and memory.
- `const std::string&` costs $\Theta(1)$ and promises not to modify.
- This is the default for read-only parameters of non-trivial types.

WHICH PARAMETER TYPE?

You want to...	Use	Example
read a small type (`int` , `double` , `char`)	by value	`int f(int n)`
read a large object	`const T&`	`void print(const std::string& s)`
modify the caller's object	`T&`	`void sort(std::vector<int>& v)`
allow "no object"	`T*`	`void f(Node* p)`

> Assignment 06 passes `NodeValue` by value. This is fine and simple. From now on you know the faster alternative: `const NodeValue&` .

Return by Reference

```cpp
int& at(std::vector<int>& v, int i)
{
  return v[i];
}
std::vector<int> v = {1, 2, 3};
at(v, 1) = 99;              // v is now {1, 99, 3}
```



- The function returns **the element itself**, not a copy, so the call can stand on the left side of `=`.
- This is how `v[i]`, `map[key]` and `front()` work.
- Const getters usually return `const T&`:

```cpp
const std::string& getName() const { return name; }
```



DANGLING REFERENCE

```
int& bad()
{
    int local = 5;
    return local;          // local dies here
}
int& r = bad();            // r refers to a destroyed variable
std::cout << r;            // undefined behavior
```



> **Rule:** never return a reference (or pointer) to a **local variable**. Return a reference only to something that outlives the call: a member, an element of a container, an argument.

REFERENCES IN RANGE-FOR

```cpp
std::vector<int> v = {1, 2, 3};

for (int x : v)
  x *= 2;                 // x is a COPY: v unchanged
for (int& x : v)
  x *= 2;                 // v is {2, 4, 6}

std::vector<std::string> words = {"a", "b"};
for (const auto& w : words)             // read only, no copies
    std::cout << w << " ";
```

`const auto&` is the default loop header for reading. Use `auto&` to modify, and plain `auto` only for cheap types.

Why Does the Copy Constructor Take a Reference?

```
class DoubleNode {
   public: DoubleNode(const DoubleNode& other);     // copy constructor
   public: DoubleNode(const DoubleNode&) = delete;  // as in Assignment 06
};
```



- `DoubleNode b = a;` calls the copy constructor.
- If the parameter were **by value** (`DoubleNode other`), passing it would require a copy, which calls the copy constructor, which needs a copy... **infinite recursion**.
- A reference makes no copy, so the recursion stops.
- Copying a node would also duplicate pointers (two owners, double `delete`). That is why we wrote `= delete`.

RVALUE REFERENCES AND STD::MOVE

- **lvalue**: has a name and an address (`x`, `v[0]`). **rvalue**: a temporary (`x + 1`, `std::string("a")`).
- `T&&` binds to rvalues. It lets a function **steal** resources from a temporary instead of copying.

```
std::string a(1000000, 'x');
std::string b = a;              // copy: Θ(n)
std::string c = std::move(a);   // move: Θ(1), takes the buffer
// a is valid but its content is unspecified
```



> `std::move` does not move anything. It only **casts** to an rvalue, so that the move constructor is chosen. `vector::push_back` and `return v;` use moves automatically.

QUIZ

```
int x = 1;
int& r = x;
int y = 2;
r = y;
r++;
std::cout << x << " " << y;
```



What is printed?

TEMPLATES

THE PROBLEM: ONE FUNCTION PER TYPE

```
int maxInt(int a, int b)
{
  return (a > b) ? a : b;
}
double maxDouble(double a, double b)
{
  return (a > b) ? a : b;
}
std::string maxStr(std::string a, std::string b)
{
  return (a > b) ? a : b;
}
```



- The body is **identical**, only the type changes.
- Every new type means another copy of the code, and another place for a bug.
- We want to write the algorithm **once** and let the compiler fill in the type.

FUNCTION TEMPLATE

```cpp
template <typename T>
T maxOf(const T& a, const T& b)
{
  return (a > b) ? a : b;
}

maxOf(3, 7);                          // T = int
maxOf(2.5, 1.5);                      // T = double
maxOf(std::string("a"), std::string("b"));   // T = std::string
maxOf<double>(1, 2.5);                // T given explicitly
// maxOf(1, 2.5);                     // error: T = int or double?
```



- `T` is a **type parameter**. The compiler **deduces** it from the arguments.
- `typename` and `class` mean the same in `template <...>`.

WHAT THE COMPILER DOES

- A template is **not code yet**, it is a recipe.
- For every type used, the compiler **instantiates** a separate function (`maxOf<int>`, `maxOf<double>`, ...).
- Instantiation happens at **compile time**, so there is no runtime cost compared to hand-written versions.
- The type must support every operation used in the body. `maxOf` needs `operator>`.

```
struct Point { int x, y; };
maxOf(Point{1, 2}, Point{3, 4});   // error: no operator> for Point
```



> The error appears **inside the template body** and is often long. Concepts (a few slides ahead) make it readable.

TEMPLATES AND REFERENCES TOGETHER

```cpp
template <typename T>
void swapValues(T& a, T& b)
{
  T t = a; a = b; b = t;
}

int x = 1, y = 2;
swapValues(x, y);              // T = int

std::string s = "a", u = "b";
swapValues(s, u);              // T = std::string
```

- `const T&` for read-only, `T&` for modifying: the same rules as for ordinary functions.
- Passing `T` by value in a template would copy a possibly huge type. Prefer `const T&`.

NON-TYPE PARAMETERS

```cpp
template <typename T, std::size_t N>
T sumArray(const T (&a)[N])        // reference to an array of N elements
{
   T s{};                          // zero for numbers
   for (std::size_t i = 0; i < N; i++) s += a[i];
   return s;
}

int    a[] = {1, 2, 3, 4};
double b[] = {0.5, 1.5};
sumArray(a);      // 10, N deduced as 4
sumArray(b);      // 2.0, N deduced as 2
```



> `N` is a **value** known at compile time. A reference to an array keeps its size, a plain pointer would lose it. `std::array<T, N>` is built the same way.

Class Template: a Generic Node

```cpp
template <typename T>
class Node {
    public: T       value;
    public: Node<T>* nextPtr;

    public: Node(T v) : value(v), nextPtr(nullptr) {}
};
// ...
Node<int>*        a = new Node<int>(5);
Node<std::string>* b = new Node<std::string>("hello");
delete a;
delete b;
```

- Class templates need the type **written explicitly**: `Node<int>`.
- `Node<int>` and `Node<std::string>` are **two unrelated classes**.
- The `classNodeT<T>` from Assignment 05 is exactly this.

CLASS TEMPLATE: STACK<T>

```
template <typename T>
class Stack {
   private: std::vector<T> data;
   public:  void push(const T& x) { data.push_back(x); }
   public:  void pop()            { data.pop_back(); }
   public:  const T& top() const  { return data.back(); }
   public:  bool empty() const    { return data.empty(); }
};

Stack<std::string> s;
s.push("a");  s.push("b");
std::cout << s.top();            // b
```



> A template may use **another template** inside (`std::vector<T>`). This is how the whole standard library is built.

WHERE DO TEMPLATES LIVE?

- The compiler must **see the whole body** when it instantiates a template.
- So template definitions go into the **header** (`.hpp`), not into a separate `.cpp`.
- If you put the body in a `.cpp`, you get a linker error: `undefined reference to Stack<int>::push`.
- Every `.cpp` that includes the header may instantiate the same template. The linker merges the identical copies.

> Rule: **declaration and definition of a template in the same header.**

Concepts (C++20): Constraints on T

```cpp
#include <concepts>

template <std::totally_ordered T>          // T must support < > <= >= == !=
T maxOf(const T& a, const T& b)
{
  return (a > b) ? a : b;
}

maxOf(3, 7);                  // OK
maxOf(Point{1, 2}, Point{3, 4}); // error: constraints not satisfied
```



- A concept states **what T must be able to do**, in the signature.
- The error is reported at the **call**, with a short reason, not deep inside the body.
- Other useful ones: `std::integral`, `std::floating_point`, `std::copyable`, `std::regular`.

Templates You Already Use

```
auto add = [](auto a, auto b) { return a + b; };   // generic lambda
add(1, 2);                    // 3
add(1.5, 2);                  // 3.5
add(std::string("a"), std::string("b"));   // "ab"

std::visit([](auto x) { std::cout << x << " "; }, nodeValue);
```



- `auto` parameters make the lambda a **template**: the compiler generates one version per type used.
- `std::variant<int, double, std::string>`, `std::optional<T>`, `std::list<T>`, `std::vector<T>` are all class templates.
- `std::visit` calls your generic lambda once for every alternative of the variant.

STANDARD

Containers

WHY USE THE LIBRARY?

In Assignment 06 you built a doubly linked list with `new`, `delete` and pointer repairs. Then you replaced it by **one** `std::list`.

- Tested by millions of programs, no leaks, no dangling links.
- Optimized by experts, and generic: `std::list<T>` works for any `T`.
- Every container has a **documented time complexity** for each operation.

Family	Containers
Sequence	`array`, `vector`, `deque`, `list`, `forward_list`
Ordered associative (tree)	`set`, `map`, `multiset`, `multimap`
Unordered (hash)	`unordered_set`, `unordered_map`
Adapters	`stack`, `queue`, `priority_queue`

STD::ARRAY AND STD::VECTOR

```cpp
#include <array>
#include <vector>

std::array<int, 3> a = {1, 2, 3};     // fixed size, on the stack
std::vector<int> v = {5, 3, 8};       // dynamic size, on the heap

v.push_back(1);                  // amortized Θ(1)
v.size();                      // 4
v[0];  v.at(0);                  // at() checks the range
v.pop_back();
v.clear();
```



> `vector<T>` is the **default** container. It is a dynamic array: contiguous memory, fast access `v[i]` in $\Theta(1)$, fast append, slow insert in the middle $\Theta(n)$.

std::array vs a Raw Array

	`int a[3]`	`std::array<int, 3> b`
size	you must remember it separately	`b.size()`
passed to a function	decays to a pointer (the size is lost)	keeps its type and size
copying	`a2 = a;` does not compile	`b2 = b;` copies the elements
comparison	`a == a2` compares **addresses**	`b == b2` compares **elements**
returned from a function	impossible	possible
range checking	none	`b.at(5)` throws an exception
iterators and `<algorithm>`	pointers only	`b.begin()`, `b.end()`, `std::sort(b.begin(), b.end())`

> Same memory layout, same speed. `std::array` is a raw array wrapped in a class.

VECTOR: SIZE, CAPACITY AND INVALIDATION

- `size()` = number of elements, `capacity()` = allocated cells. When full, `push_back` allocates a **bigger block** (about 2×) and moves the elements. That is why append is amortized $\Theta(1)$.
- `v.reserve(n)` allocates in advance and avoids the reallocations.

```cpp
std::vector<int> v = {1, 2, 3};
int& first = v[0];
v.push_back(4);          // may reallocate: the old block is freed
first = 99;              // undefined behavior: dangling reference
```



> Any reference, pointer or iterator into a vector may be **invalidated** by `push_back`. References from the first part of this lecture are valid only as long as the object lives where it was.

STD::LIST AND STD::DEQUE

```cpp
#include <list>
#include <deque>

std::list<int> l = {1, 2, 3};      // doubly linked list (your Assignment 06)
l.push_front(0);   l.push_back(4); // Θ(1) at both ends
l.pop_front();     l.remove(3);    // remove(x) deletes ALL elements equal to x
// l[1];                          // error: no random access

std::deque<int> d = {1, 2, 3};     // double-ended queue
d.push_front(0);   d.push_back(4); // Θ(1) at both ends
d[2];                              // Θ(1) random access, unlike list
```



> `list` inserts or erases in $\Theta(1)$ **at a known position**, but finding the position costs $\Theta(n)$ and every node is a separate allocation. In practice `vector` or `deque` is usually faster.

ITERATORS

An **iterator** is a generalized pointer: `*it` is the element, `++it` moves to the next one. Every container provides `begin()` (first) and `end()` (**one past** the last).

```cpp
std::vector<int> v = {5, 3, 8};

for (auto it = v.begin(); it != v.end(); ++it)
    std::cout << *it << " ";

auto it = std::find(v.begin(), v.end(), 3);     // <algorithm>
if (it != v.end())
    v.erase(it);                        // removes only that element
```



> `find` returns `end()` when nothing is found. The same code works for `vector`, `list`, `set`, ...: iterators connect containers with algorithms.

ADAPTERS: STACK, QUEUE, PRIORITY_QUEUE

```cpp
#include <stack>
#include <queue>

std::stack<int> s;
s.push(1); s.push(2);
s.top();       // 2
s.pop();       // LIFO

std::queue<int> q;
q.push(1); q.push(2);
q.front();     // 1
q.pop();       // FIFO
```



```cpp
std::priority_queue<int> pq;
pq.push(3); pq.push(9); pq.push(5);
pq.top();      // 9 (largest first)
pq.pop();      // Θ(log n)
```



An adapter **wraps** another container (`deque` by default) and exposes a smaller interface. `priority_queue` is a binary heap (the heapsort structure).

STD::MAP AND STD::SET

```cpp
#include <map>
#include <set>

std::map<std::string, int> freq;
for (const std::string& w : words) freq[w]++;      // missing key starts at 0

for (const auto& [word, n] : freq)                 // sorted by key
    std::cout << word << ": " << n << "\n";

std::set<int> s = {5, 1, 5, 3};                    // {1, 3, 5}: no duplicates
s.contains(3);                                     // C++20
```

- Balanced search trees: insert, erase and find in $\Theta(\log n)$, iteration in **sorted order**.
- `m[key]` **inserts** the key if it is missing. To only look, use `m.find(key)` or `m.contains(key)`.

UNORDERED_MAP AND UNORDERED_SET

```cpp
#include <unordered_map>
#include <unordered_set>

std::unordered_set<int> seen;
bool hasDuplicate(const std::vector<int>& A)      // Θ(n) expected, Θ(n) space
{
    for (int x : A)
    {
        if (seen.contains(x)) return true;
        seen.insert(x);
    }
    return false;
}
```

- Hash table: insert, erase, find in $\Theta(1)$ **expected**, $\Theta(n)$ worst case. No order.
- This is the "hash set" row from the time–space trade-off table in Lecture 15.

Complexity Cheat Sheet

Container	[i]	Insert / erase middle	Insert at end	Find value
vector	$\Theta(1)$	$\Theta(n)$	$\Theta(1)$ amortized	$\Theta(n)$
deque	$\Theta(1)$	$\Theta(n)$	$\Theta(1)$ (both ends)	$\Theta(n)$
list	no	$\Theta(1)$ at an iterator	$\Theta(1)$ (both ends)	$\Theta(n)$
set, map	no	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)$
unordered_*	no	$\Theta(1)$ expected	$\Theta(1)$ expected	$\Theta(1)$ expected

> These are the same $T(n)$ and Θ you analyze in this course. The table is your tool to **choose** a container.

ALGORITHMS ON CONTAINERS

```cpp
#include <algorithm>
#include <numeric>

std::vector<int> v = {5, 2, 9, 1};

std::sort(v.begin(), v.end());                    // {1, 2, 5, 9}
std::sort(v.begin(), v.end(), [](int a, int b) { return a > b; });  // descending

auto big = std::count_if(v.begin(), v.end(), [](int x) { return x > 3; });
int sum  = std::accumulate(v.begin(), v.end(), 0);   // 17
bool has = std::binary_search(v.begin(), v.end(), 5); // needs ascending order!
```

`sort` is $\Theta(n \log n)$, `binary_search` is $\Theta(\log n)$. The algorithms are **templates** that work on iterators, so one `sort` serves `vector`, `deque` and `array`.

CONTAINERS OF CONTAINERS

```cpp
std::map<std::string, std::vector<std::string>> graph;   // adjacency list

graph["A"].push_back("B");
graph["A"].push_back("C");
graph["B"].push_back("C");

for (const auto& [node, neighbours] : graph)
{
    std::cout << node << ":";
    for (const auto& n : neighbours) std::cout << " " << n;
    std::cout << "\n";
}
```

- Templates compose: `map<string, vector<string>>` is one type built from three templates.
- The loop uses `const auto&` twice: **no element is copied**.
- This is the standard representation of a graph in the algorithms you will meet next.

Which Container?

You need...	Use
a sequence, no special needs	`vector`
fixed size known at compile time	`array`
fast insert at the front and the back	`deque`
stable references and insert at a known position	`list`
unique, **sorted** keys; range queries	`set`, `map`
fastest lookup by key, order irrelevant	`unordered_set`, `unordered_map`
last-in-first-out, first-in-first-out, largest first	`stack`, `queue`, `priority_queue`

> Start with `vector`. Change the container only when the analysis of $T(n)$ says so.

SUMMARY

- A **reference** is an alias: `T&` modifies, `const T&` reads without copying. Never return a reference to a local variable.
- **Templates** write the algorithm once: `template <typename T>`. The compiler instantiates one version per type, so they live in headers. Concepts make errors readable.
- The **standard containers** are class templates with documented complexity. Default to `vector`, choose others by the operations you need.
- **Iterators** and `<algorithm>` connect any container with `sort`, `find`, `count_if`, ...

EXERCISES

1. Write `template <typename T> void printAll(const std::vector<T>& v)` that prints all elements.
2. Write `template <typename T> int countEqual(const std::vector<T>& v, const T& x)`.
3. Why does `for (int x : v) x = 0;` not clear the vector? Fix it.
4. Read words and print each distinct word with its count, sorted alphabetically (`map`) and then in the order of first appearance (`vector` + `unordered_map`).
5. Rewrite `hasDuplicate` with `sort` instead of a hash set. What are $T(n)$ and $S(n)$?

THANK

You